```
docker run -d \
  --name jenkins \
  -p 8080:8080 \
  -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  jenkins/jenkins:lts
```

To check out code from a **private** GitHub repository using the official **`jenkins/jenkins:lts`** Docker image, you need to spin up the container and provide Jenkins with authenticating credentials. The image already comes with `git` pre-installed, so you do not need to modify the image itself. [1, 2]

You can accomplish this using either **HTTPS (Personal Access Token)** or **SSH (Deploy Keys)**. [1, 2]

Step 1: Run the `jenkins/jenkins:lts` Container

Start by running the official LTS container: [1]

bash

```
docker run -d \
  --name jenkins \
  -p 8080:8080 \
  -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  jenkins/jenkins:lts
```

Use code with caution.

Retrieve your initial administrator password by running `docker logs jenkins`, navigate to `http://localhost:8080`, and finish the initial setup wizard by installing the recommended plugins (which includes the Git plugin*).* [1, 2]

Step 2: Choose Your Authentication Method

Option A: Authenticate via HTTPS (Personal Access Token) – *Easiest*

1. **Generate a PAT on** GitHub: Go to **Settings** > **Developer Settings** > **Personal Access Tokens** > **Tokens (classic)**. Click **Generate new token** and check the **`repo`** scope box. Copy the generated token. [1, 2]
2. **Add Credentials to Jenkins**: Go to **Manage Jenkins** > **Credentials** > **System** > **Global credentials** > **Add Credentials**.

- **Kind:** Username with password
- **Username:** Your GitHub username
- **Password:** *Paste your GitHub Personal Access Token*
- **ID:** `github-token` (or a descriptive name) [1, 2]

3. **Configure your Pipeline/Job**: Set the Repository URL to the HTTPS variant (e.g., **https://github.com**) and select your newly created credential from the dropdown menu. [1]

Option B: Authenticate via SSH (Deploy Keys) – *More Secure*

1. **Generate SSH Keypair**: Open your terminal and generate an SSH key pair: [1]

bash

```
ssh-keygen -t ed25519 -C "jenkins-deploy-key" -f ./jenkins_key
```

Use code with caution.

2. **Add Public Key to GitHub**: Go to your GitHub Private Repository > **Settings** > **Deploy keys** > **Add deploy key**. Paste the contents of `jenkins_key.pub`. [1]
3. **Add Private Key to Jenkins**: Go to **Manage Jenkins** > **Credentials** > **System** > **Global credentials** > **Add Credentials**.

- **Kind:** SSH Username with private key
- **Username:** `git`
- **Private Key:** Select **Enter directly** and paste the entire contents of your private key file (`jenkins_key`).
- **ID:** `github-ssh` [1, 2]

4. **Configure your Pipeline/Job**: Use the SSH repository URL (e.g., `git@github.com:your-username/your-repo.git`) and link your SSH credentials. [1]

Step 3: Implement Checkout in your Jenkinsfile

Once your credentials are configured, you can call them seamlessly inside a Jenkins Pipeline script using the `checkout` or `git` syntax: [1]

groovy

```
pipeline {
   agent any
   stages {
      stage('Checkout Code') {
         steps {
            // If using Option A (HTTPS)
            git url: 'https://github.com',
               branch: 'main',
               credentialsId: 'github-token'

            // OR If using Option B (SSH)
            // git url: 'git@github.com:your-username/your-
repo.git',
            //     branch: 'main',
            //     credentialsId: 'github-ssh'
         }
      }
   }
}
```

Use code with caution.

If you are running into specific issues, let me know:

- Which **authentication type** (HTTPS token or SSH key) you prefer to use. [1, 2]
- Any **error codes** or console output you are seeing during checkout. [1, 2]